CSC336 · WEB TECHNOLOGIES

Assignment 3 Interactive Tesla Vehicles

Making the Assignment 2 storefront interactive with Fetch API, JavaScript cart logic, array methods, and regular-expression validation.

Student: Hussnain Ali · Reference site: Tesla

01 · DYNAMIC DATA

Products from products.json

The page starts with an empty product container. On load, **fetch("products.json")** retrieves the catalog and JavaScript creates each card. This keeps product data separate from the HTML.

15

Vehicles loaded

Model S, Model 3, Model X, Model Y, Cybertruck and more.

0

Hardcoded cards

The listing is rendered from JSON rather than repeated markup.

Core implementation

```
const response = await fetch("products.json");
const products = await response.json();
renderProducts(products);
```

Validation evidence

HTTP 200 · products.json

15 vehicles rendered

No console errors

02 · CART LOGIC

Working cart and running total

Each product button adds an object to the cart array. The cart page uses **map()** to render rows, **filter()** to remove items, and **reduce()** for the item count and total. localStorage keeps the cart available between pages.

1 → 2

Quantity update

Changing Model S quantity recalculated its line total immediately.

$179,980

Calculated total

The total was recalculated from $89,990 × 2.

Core implementation

```
cart = cart.filter(item => item.id !== id);
total = cart.reduce((sum, item) =>
  sum + item.price * item.quantity, 0);
```

Interaction evidence

- Add to Cart updates badge
- Quantity controls work
- Remove action works

03 · VALIDATION

Checkout validation and final files

The checkout form blocks submission until every field passes its rule. Email and phone use regular expressions, while clear inline messages tell the user how to correct invalid data.

Regex rules

Email: standard address pattern
Phone: **03XX-XXXXXXX**
Name: letters and spaces
Address: minimum 8 characters

Invalid test result

Four invalid values displayed four specific errors and the order was not submitted.

Deliverables

```
index.html · cart.html · checkout.html
script.js · styles.css · products.json
images/ · README.md
```

Requirement checklist

✓ Plain JavaScript, no React
✓ Fetch API product loading
✓ In-memory cart with page persistence
✓ map(), filter(), and reduce()
✓ Regex validation with blocked submission

The project is served locally for Fetch API support. Opening index.html through a local HTTP server is recommended.